Homelumina

Daniel Fitz
Filmon Mengisteab
John Suli
Roy Hung

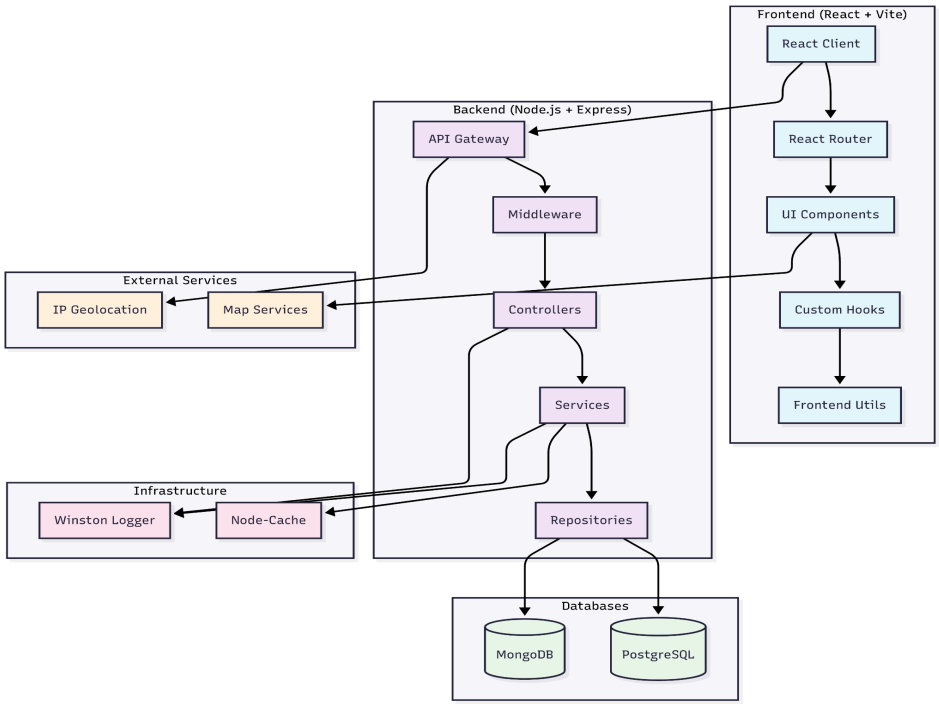
Introduction

Nowadays, people often relocate across the United States in search of better job opportunities or improved living environments. For students in the MCIT program, many of us may have to relocate for post-graduation jobs or summer internships. However, priorities can vary—single individuals may prioritize areas with higher salaries, while people with families might seek safer neighborhoods with ample access to local hospitals. As a result, choosing the ideal location can be challenging due to the numerous factors involved. To address this, we developed a user-friendly website that allows users to explore income, safety, and health data at the ZIP code level, helping them make more informed decisions when selecting a new home.

The application enables users to view a wide range of information on each ZIP code through a search feature. For each ZIP code result, users can view a comprehensive data summary, including the number of hospitals, local median income, and more. Additional filters—such as income thresholds or minimum population—can be applied to match the user's criteria. The application also offers the ability to conduct in-depth analyses of U.S. cities, allowing users to compare statistics between two cities directly within the interface. The detailed information provided helps users make more informed choices when selecting their future home.

Architecture

The simplified architectural overview is as shown below. The link for a simplified overall architecture can be found here (link).



Frontend (Client)

Our frontend is a basic client-side rendered (CSR) application built on React to compose its UI. To support our application's needs, we also added these complementary libraries. A block diagram of the front end architecture can be seen in Appendix A:

- **React Router** - routing library that handles client-side navigation—including transferring of state through query parameters—between our many pages.
- **Tanstack Query** - data fetching library that we use to query our server API endpoints, as it manages complexities around data fetching like loading and error states, caching and cache invalidation, refetching, etc.
- **Shadcn** - component library (built from Tailwind CSS and Radix UI) that gives us UI building blocks like inputs, cards, and sliders for increased velocity.
- **Lucide React** - icon library imported as React components with 1000+ icon options
- **Leaflet** - renders a map with latitude/longitude pins, which we use in the search results page to display a list of ZIP codes in "Map View" mode.
- **Firebase Authentication** - Firebase Authentication SDK to authenticate users. It supports authentication using email and password, as well as SSO providers like Google and GitHub.
- **Recharts -** library used for building line charts, namely the median listing price trends chart for a given zip code.

Backend (Server)

We have included a breakdown of our backend server below. The simplified architectural overview of the backend is shown in the block diagram in Appendix A.

- **Runtime:** Node.js with Express.js
- **Database**: PostgreSQL (primary), MongoDB (user management)
- **Caching**: In-memory cache with node-cache
- **API Documentation**: Swagger/OpenAPI
- **Logging**: Structured logging with winston
- **Validation**: Custom middleware with Joi
- **Error Handling**: Centralized error handling middleware

The request flow architecture is in this sequence diagram (flow diagram), the detailed sequence diagram follows in each page.

Architecture Patterns:

- **MVC Pattern**: Controllers → Services → Repositories
- **Repository Pattern**: Data access abstraction
- **Middleware Pattern**: Request validation, error handling, logging
- **Caching Strategy**: Multi-level caching for performance

• Home Page

The home page serves as the starting point for users to begin their search on Zip code. We implemented a search bar with auto-complete suggestions to enhance the user experience. The home page also includes a login/sign up feature, allowing users to sign/sign up into the application. Additionally, users can access quick results by selecting options such as Best for Families within the Quick Search category section. Lastly, the home page highlights three featured cities. These search results are tailored to each user IP address. For a user in Georgia IP address, the query in this section will return suggested cities from Georgia as well. We also implemented a cache feature to store the query result, so the application can just retrieve the result from the cache.

• Search Results Page

When a user selects a city or state in the home page search bar, we direct them to the search results page, where we show a list of ZIP codes within that city or state—with basic summary data for each ZIP code included. This list can be viewed in “List View”, “Map View”, or “Analytics View” mode.

The search results page serves as a bridge where the user can narrow down the list of ZIP codes within a city or state into a smaller and smaller subset using filters that they care about: min/max

housing price, min/max income, min/max population, min police departments, min fire stations, min childcare centers, etc. The goal of this page is to help the user narrow down their search to specific ZIP codes—at which point they can click "View Details" to get even more detailed data in the location details and comparison pages.

• Location Details Page

If the user selects a zipcode from the search result page it will be redirected to the Location Details page. This page will show a summary for the specific location such as population, median income, home price, health score, obesity rate, asthma rate, depression rate, hospitals, police stations, fire stations, childcare centers, similar locations (zipcodes), data quality metrics, overall health grade and affordability grade. A quick actions tool will allow the user to compare the location to another zipcode and redirect the user to the Comparison Page.

• Comparison Page

When users are viewing information on the location details page, they can use the quick action tool to navigate to the Comparison page. This page displays summary data of two cities side by side, making it easy for users to compare them. Additional analyses, such as city home price growth rates, are also included. The page also provides redirect links to other cities that are similar to the ones being viewed, offering users more options for exploration and comparison. Lastly, a toggle feature allows users to hide unimportant information from the page, reducing the amount of information presented on the page.

Data

Description

To gather detailed background information for each location, we use multiple data sources. The primary table in our database is zipcode.csv, retrieved from ZipCode.org, which provides all U.S. ZIP codes in a single CSV file. The second source is historical monthly housing inventory data from Realtor.com. This dataset contains over 2 million rows of historical housing data, including information such as the number of housing units in inventory and the average monthly home price for each U.S. ZIP code across the 10 year period.

Datasets on hospitals, fire stations, police stations, and childcare centers were retrieved from the Homeland Infrastructure Foundation-Level Data (HIFLD), which is maintained by the U.S. Department of Homeland Security. These dataset contain information on department id , zip code, longitude, latitude, and more information for the facility. The dataset on health measures is hosted by the U.S. Centers for Disease Control and Prevention (CDC) and provides exactly 40 measures of health metrics, detailing their prevalence within the populations of each U.S. ZCTA (ZIP Code Tabulation Area). Lastly, the household income dataset and population is sourced

from the American Community Survey conducted by the U.S. Census Bureau. This file includes various metrics on household income for each geographic area. The table below shows summary statistics for each dataset.

Dataset Name	Data Source	File Size	Row Count	Column Count
Local market.csv	Realtor.com	704 MB	2999230	46
Zipcode.csv	zipCode.org	4.4 MB	42736	13
Hospital.csv	Homeland Infrastructure Foundation	2.06 MB	8341	34
Firefigher_Station.csv	Homeland Infrastructure Foundation	1.7 MB	27088	26
Police_Station.csv	Homeland Infrastructure Foundation	10.3 MB	23487	38
ChildCareCenter.csv	Homeland Infrastructure Foundation	4 MB	132218	27
Healthmeasure.csv	US Center of Disease Control	235 MB	1236338	20
Household_income.csv	US Census	10 MB	33774	170

Table 1: Data Set Summary

Preprocessing

Each dataset came in the form of CSV files. We then load the files into pandas dataframe to begin the preprocessing steps. For each dataset, we thoroughly scanned all entries for incorrect ZIP code format. Since the relations in the postgres database will be joined on the Zipcode attribute, it is necessary to ensure all Zipcode values in each dataset match the 5 digits format. In the case of household income dataset, each ZIP had a “ZTCA” prefix, meaning the 5-digit ZIP code had to be parsed from the string. For the childcare center dataset, some ZIP code values included the 4-digit extension, requiring us to extract the first 5 numeric digits. Therefore, all ZIP code entries across the 9 datasets were checked to ensure valid formats. For ZIP codes such as "00680," some entries were stored as "680." To correct this, we cast the ZIP code column in each dataset to string type and used the zfill method to pad leading zeros for entries with fewer than 5 characters. Furthermore, to ensure optimal speed when uploading the datasets into PostgreSQL, all columns in each dataset were converted to consistent data types to avoid type discrepancies within columns. ZIP code entries that did not correspond to residential ZIP codes were also removed from each dataset to prevent violations of foreign key constraints when uploading to the PostgreSQL database. To reduce the amount of information that is stored in the database, we removed columns that were irrelevant to our application. For example, we dropped the pending_ratio column from the Realtor dataset. Reducing the number of unnecessary columns also improves the speed of uploading the dataset into the postgres database.

Database

Using post-processed datasets, we created 9 tables in the postgres database. During the preprocessing steps, we had scanned through each entry to ensure meeting the foreign key constraint during the data ingestion step. As each table contains a ZIP code attribute, we use ZIP code as the attribute in our query to join several tables together to get the comprehensive information on a location.

Table Name	Number of Instance
ChildCareCenters	132165
City	29784
FirefigherStations	27071
HealthMeasure	177886
Hospitals	8334
HouseholdIncome	33772
LocalMarket	2999208
PoliceStations	23479
Zipcode	42735

Table 2: Number of Instance For Each Table

The relational schema of each table is shown below:

Relational Schema:

- ChildcareCenters(Id, ZipCode, Latitude, Longitude),
  Primary key (Id), Foreign key ZipCode references ZipCode(ZipCode)
- City(Name, State)
  Primary key (Name, State), Foreign key (Name, State) references ZipCode(City, State)
- FirefighterStations(Dptid, ZipCode, NumofStations, ActivePersonnels)
  Primary key (Dptid), Foreign key ZipCode references ZipCode(ZipCode)
- HealthMeasure(Measure, ZipCode, Ratio, TotalPopulation)
  Primary key (Measure, ZipCode), Foreign key ZipCode references ZipCode(ZipCode)
- Hospitals(ID, ZipCode, Latitude, Longitude, NumofBeds, TraumaLevel, HasHelipad)
  Primary key (ID), Foreign key ZipCode references ZipCode(ZipCode)
- HouseholdIncome(GeoId, Zipcode, MeanIncome)
  Primary key (GeoID), Foreign key ZipCode references ZipCode(ZipCode)
- LocalMarket(MonthDate, Zipcode, Latitude, Longitude, ActiveListingCount, MedianListingPrice)
  Primary key (MonthDate, Zipcode) , Foreign key ZipCode references ZipCode(ZipCode)
- PoliceStations(Dptid, ZipCode, PartTimeOfficers, FullTimeOfficers, TotalActiveOfficers)
  Primary key (Dptid) , Foreign key ZipCode references ZipCode(ZipCode)

- ZipCode(ZipCode, Latitude, Longitude, Population, City, State)
  Primary key (ZipCode)

Normal Form
Our relations are all in BCNF, as, in each table, all functional dependencies are implied by the keys. Several tables—such as ChildcareCenters, FirefighterStations, and others—can contain duplicate ZipCode values because a single ZIP code may correspond to multiple facilities. Therefore, ZipCode does not functionally determine other attributes on its own. Below are the functional dependencies for each relation:
ZipCode: ZipCode → Latitude, Longitude, Population, City, State
ChildcareCenters: ID → ZipCode, Latitude, Longitude
FirefighterStations: Dptid → ZipCode, NumofStations, ActivePersonnels
HealthMeasure: Measure, ZipCode → Ratio, TotalPopulation
Hospitals: ID → ZipCode, Latitude, Longitude, NumofBeds, TraumaLevel, HasHelipad
HouseholdIncome : GeoId → Zipcode, MeanIncome
LocalMarket: MonthDate, Zipcode →Latitude, Longitude, ActiveListingCount, MedianListingPrice
PoliceStations: Dptid → ZipCode, PartTimeOfficers, FullTimeOfficers, TotalActiveOfficers

Queries

We implemented 11 queries (even though we wrote 67 total queries which are found in the server/queries folder) for this application. The five queries below are used as example query to show how we retrieve information from the database. The code of these 5 queries (along with 6 others) are shown in appendix C:

- **Query 1 GET /api/v1/search/zipcode-summaries (complex):** The complex query underlying this endpoint is used to populate the list of ZIP codes displayed in the search results page. Because the search results page includes so many filters, this query accepts 17 query parameters. The reason why this query is complex comes from the fact that it aggregates and joins almost every single table in our database on every ZIP code within a city or state. In the worst case, we are aggregating and joining up to 2661 ZIP codes (for the entire state of "TX").
- **Query 2: GET /api/v1/analytics/city-aggregate-comparison (complex):** This complex query is used to aggregate all housing, police, childcare, hospitals and other data for all ZIP codes within a city. We used this query in order to populate all necessary data for the location details page and also to populate the city in the comparison page. This query accepts two required parameters of city and state. We classified this query as a complex query due to the high amount of CTE that was initially used in order to aggregate data for each relation in the database. The original query requires seven custom table expressions with aggregation by zip code on several of them.

- **Query 3 GET /api/v1/analytics/similar-cities/:city/:state (complex):** This complex query is used to find two similar cities from the input city that was used in the city-aggregate-comparison API in the Comparison page. We used this query to calculate the similarity score of other cities which is measured as the sum of absolute difference between different attributes with the base city. The user can then click on the similar city to be redirected to look at more information about this similar city.
- **Query 4 GET api/v1/analytics/similar-zipcodes/:zipcode (complex):** This complex query returns the 4 most similar ZIP codes for the given ZIP code in the Locations Detail Page based on the similarity score which is measured as the sum of absolute difference between different attributes. It shows the result inside the Quick Actions component for the user to quickly glance and type if needed.
- **Query 5 GET api/v1/analytics/city-growth-rate/:city/:state:** This query returns the 3 year growth rate for the given city and state. This is used on the Comparison page to provide additional real estate analysis. CTE was used to retrieve 2025 and 2022 average housing prices for the input city/state. The two CTE are then joined to calculate the change in housing price across the three years.

Performance Evaluation

Endpoint for query	Timing before optimization	Timing after optimization
Query 1: zipcode summary	Worst case: 2 minutes (querying for state of "TX" with most ZIP codes)	Worst case: ~1600ms Average case: <1 second
Query 2: aggregate -comparison	Worst case: 4 seconds (ex. "Houston, TX")	Worst Case: ~400 ms
Query 3: similar cities	Worst case: 16 seconds (ex. "Houston, TX")	Worst Case: ~500 ms
Query 4: similar zip code	Worst case: ~8 seconds (ex. "19104")	Worst Case: ~600 ms

An overarching optimization we performed was adding an index on ZIP code for every table that did not already include ZIP code as part of its primary key. This change was beneficial because almost every query involves joining on ZIP code. As a result, the API time for Query 2 was reduced from 4 seconds to just 0.4 seconds when joining across seven tables with aggregation.

For the Query 1 API, we further optimized performance by introducing materialized views. The localmarket table contains over 2.9 million records, making it time-consuming to compute the average monthly housing price for all relevant ZIP codes on every query. To address this, we

wrote a SQL query to compute the average housing price for all U.S. ZIP codes and stored the result in a materialized view named market2025_view. We then created an index on ZIP code within this view to enable faster data access. With the materialized view and index in place, we updated the DML for the Query 1 API endpoint to retrieve housing prices directly from the materialized view. This reduced the query time from over 2 minutes to approximately 1 second. For the code used to create the materialized view that was used in this query. Please see Appendix C1 for the code to construct the materialized view.

For query 3 and query 4 API, both of these SQL query will find the most similar zip code or city by comparing with all other zip code or city within the US. The list of non-input city or zipcode will serve as our reference table to use in the similarity calculation. Therefore, both of these queries involve a large amount of aggregation to retrieve the location information -such as the number of hospitals per location or population- for each zipcode/city. To calculate the similarity score between input and reference table cross join is done to calculate the difference of all other US zipcode/city compared to the current input. Since the reference table will not change between different execution, it is very resource intensive to repeat the same aggregation for non input zip/city in the US. Hence, we implemented materialized views to store the reference result for all US zip and city in the database. Hence, this has drastically reduced the amount of scanning each table has to do before calculating the similarity score. For the materialized view in similar Zipcode API, we have created an index on zipcode for faster retrieval. For the materialized view on similar city API, we created an index on both (city, state) since these two conditions are used together to get the input city. This had reduced the run time from roughly 15 seconds for similar city down to 0.5 second. As the data used in API 2 overlap with simiar_city_view, we have updated query 2 to retrieve the result from the view for faster performance.

Technical Challenges

1. Deciding on the idea to use the app was initially difficult. We started by looking into available data that could easily be pre-processed and joined with other data first which meant that we had to consider different ideas. Our first idea was creating a food-related app, but we realized that string manipulations and processing primary and foreign keys was challenging. Next, we thought of creating a npm Package Manager that would track all the latest trends from the npm library, but the available data was not enough to fulfill the project requirements. Finally, we decided on a Zillow like app with information about different locations for those interested in moving to a new city or neighborhood. Since all locations had a standard zipcode, pre processing the data and creating primary and foreign keys was easy to manage. Additionally, there were multiple sources of data available that were related to the main idea.
2. We experienced difficulties when cleaning our data and populating our database. Specifically, we ran into foreign key violations where different tables had invalid foreign key violations since they were referring to zipcodes that weren’t present in our main

zipcode table. We solved this issue by first creating a set with all the valid zipcodes from the main table. Then, we used a simple if statement in pandas to omit the invalid zip codes before loading the data in our database.

3. Optimizing the queries for time under 1 second was difficult until we started using materialized views. The most problematic table was the localmarket table holding over ~3 million entries from the realtor data listing the median listing price of each zipcode for each month for the past 30 years. Any join or processing of this table took >1 second. We created a materialized view for this table along with indexing the zip code. In specific queries we limited the price to the latest month in a where clause.

Extra Credits

- Signup/Login experience - Secure Firebase authentication (sign-in, sign-up) using email and password, SSO (google and github). A persistent session is established for authenticated users.
- NoSQL (MongoDB) integration - Whenever a user successfully signs up with Firebase authentication a POST request creates a corresponding user record with a timestamp (createdAt) in a separate MongoDB database. Admins can retrieve all registered users via a GET users call for analytics.
- The server has Jest tests implemented which can be run using “npm run test” in the terminal (must cd to server).
- Cool Feature 1: Featured Cities are Dynamically fetched based on the user’s IP address which finds the state where the user lives and fetches the top 3 cities in that state. This is also cached in memory for faster access using local Node-Cache. If the IP address can’t be determined (for example IP is private/local host, invalid/unrecognized IP address, etc), then 3 default cities are shown.
- Cool Feature 2: The app includes an interactive React Leaflet Map that is fully functional and displays data by zipcode. Markers are custom SVG pins (with our app logo) and display detailed information upon clicking.
- Cool Feature 3: The app is fully responsive, optimized for mobile, tablet, and desktop devices.
- Cool Feature 4: It has dark mode which is automatically enabled based on the user's browser or system preference.

Appendix A1: Additional Architecture Diagram

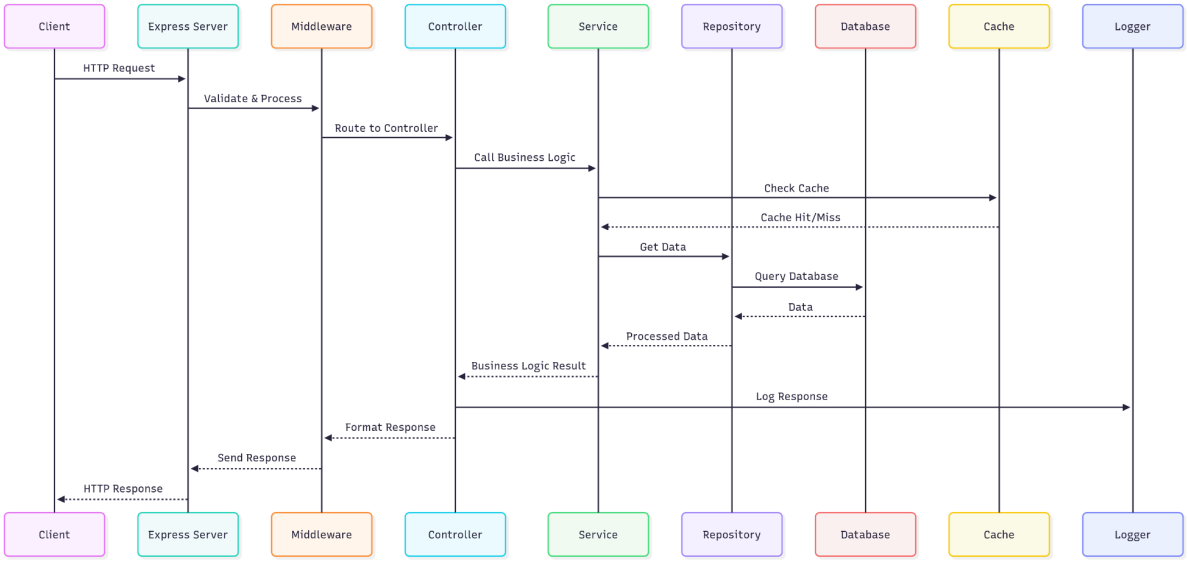


Table A1: Mermaid Chart of overall Architecture

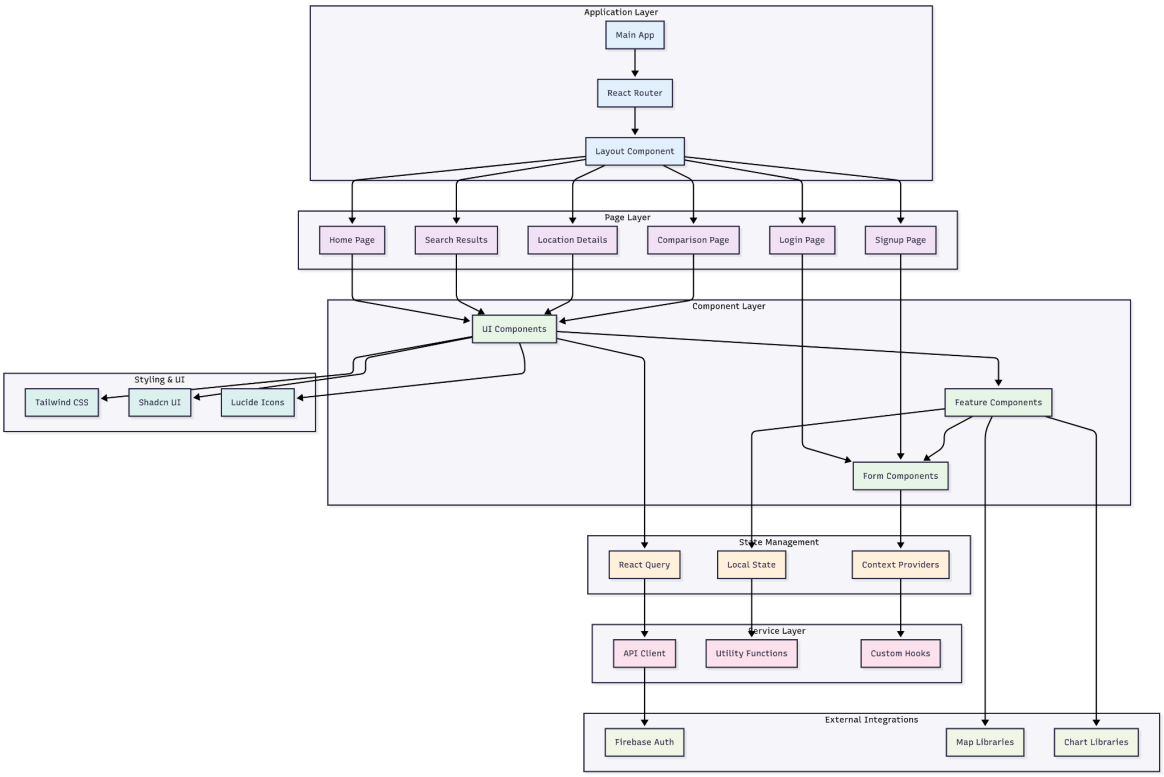


Table A2: Block Diagram of Frontend Architecture with link

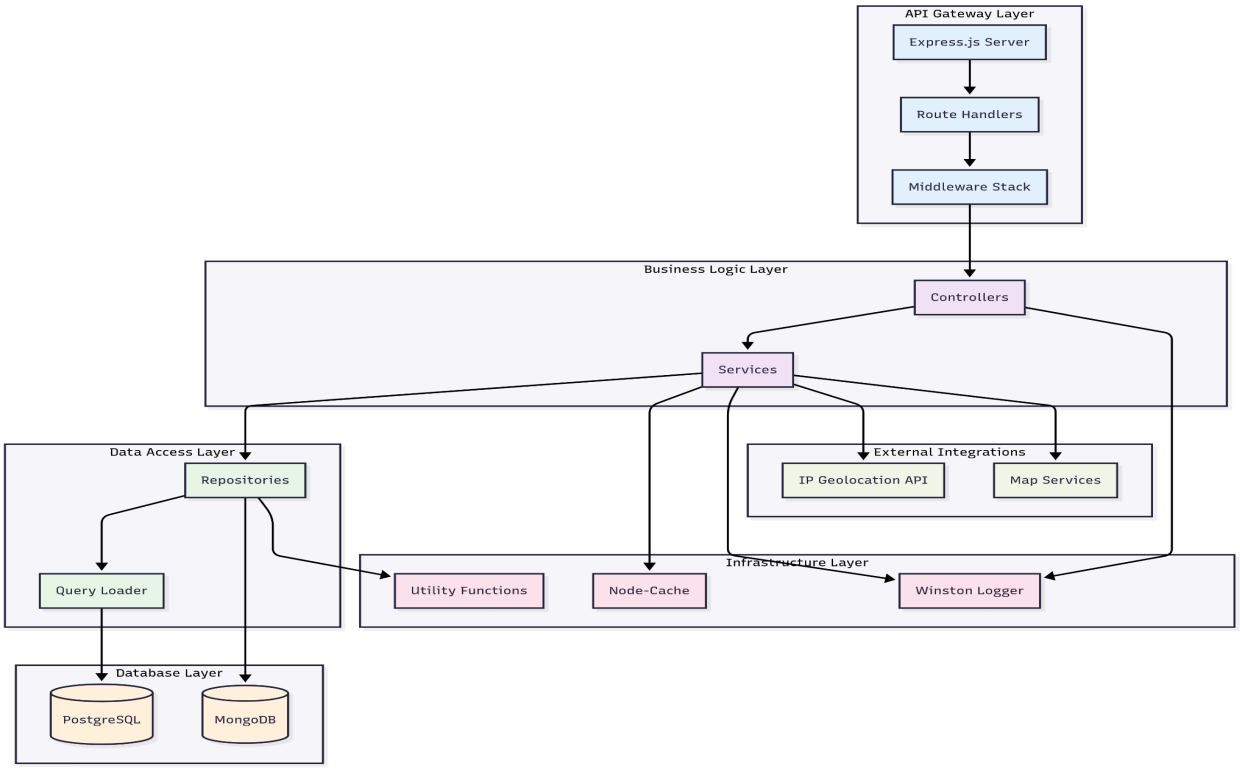


Table A3: Block Diagram of Backend Architecture (diagram).

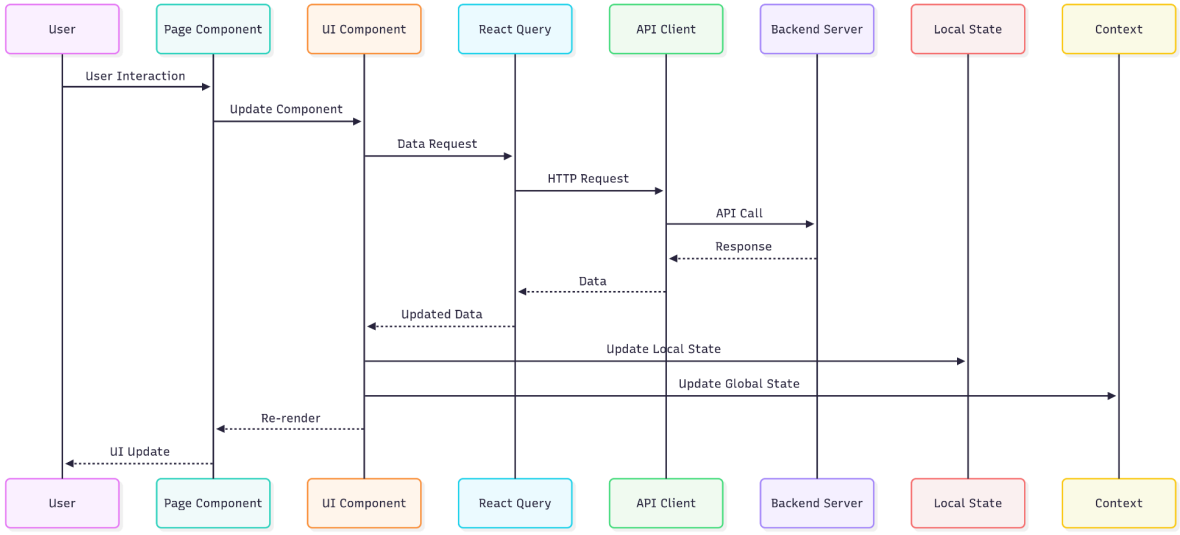
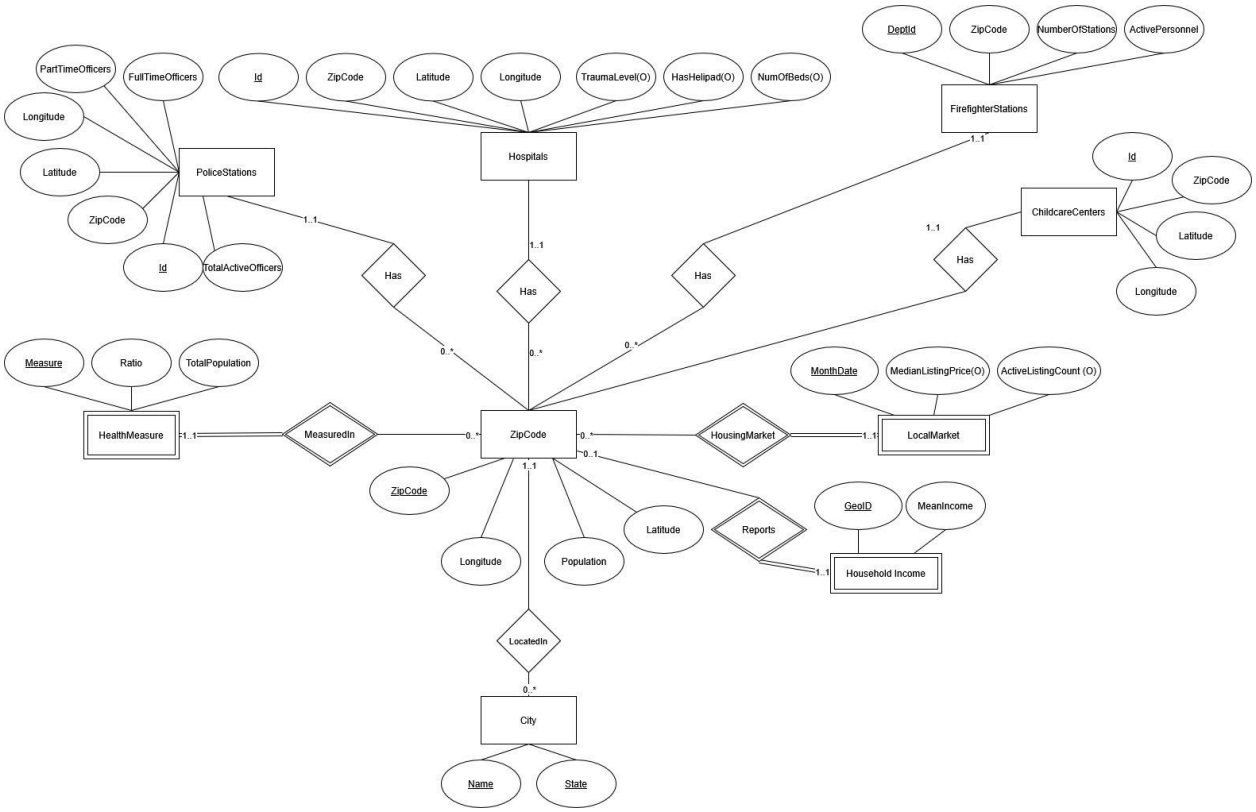


Table A4: The mermaid chart of Data Flow Architecture of UI data flow (diagram).

Appendix B: Entity Relationship Diagram



Appendix C: Example Query

Query 1 (complex): /api/v1/search/zipcode-summaries

Description: Populates the list of ZIP codes displayed in the search results page.
Pre-optimized: Running time +2 minutes
Post-optimized: Running time 400 ms - 1.6 seconds (worst case)
Optimized Query Code:

```
WITH ZipCodesInCityOrState AS (
 SELECT *
 FROM zipcode
 WHERE state = $1
   AND ($2::text IS NULL OR city ILIKE $2)
),
LatestMarket AS (
        SELECT lm.zipcode, lm.medianlistingprice
        FROM market2022_view lm
                JOIN ZipCodesInCityOrState z ON lm.zipcode = z.zipcode
    ),
IncomeStats AS (
```

```
 SELECT hi.zipcode, hi.meanincome
 FROM householdincome hi
 JOIN ZipCodesInCityOrState z ON hi.zipcode = z.zipcode
),
HealthStats AS (
 SELECT hm.zipcode, hm.ratio AS HealthRatio
 FROM healthmeasure hm
 JOIN ZipCodesInCityOrState z ON hm.zipcode = z.zipcode
 WHERE hm.measure = $3
),
PoliceStats AS (
 SELECT ps.zipcode,
        COUNT(*) AS PoliceDeptCount,
        SUM(totalactiveofficers) AS PoliceOfficerCount
 FROM policestations ps
 JOIN ZipCodesInCityOrState z ON ps.zipcode = z.zipcode
 GROUP BY ps.zipcode
),
HospitalStats AS (
 SELECT h.zipcode, COUNT(*) AS HospitalCount
 FROM hospitals h
 JOIN ZipCodesInCityOrState z ON h.zipcode = z.zipcode
 GROUP BY h.zipcode
),
FireStats AS (
 SELECT f.zipcode,
        SUM(numberofstations) AS FireStationCount,
        SUM(activepersonnel) AS FirefighterCount
 FROM firefighterstations f
 JOIN ZipCodesInCityOrState z ON f.zipcode = z.zipcode
 GROUP BY f.zipcode
),
ChildcareStats AS (
 SELECT c.zipcode, COUNT(*) AS ChildcareCount
 FROM childcarecenters c
 JOIN ZipCodesInCityOrState z ON c.zipcode = z.zipcode
 GROUP BY c.zipcode
)
SELECT
 z.zipcode,
 z.city,
 z.state,
 z.latitude,
 z.longitude,
 COALESCE(z.population, 0) AS population,
 lm.medianlistingprice AS medianprice,
 ins.meanincome AS meanincome,
 hes.healthratio AS healthratio,
```

```
 ps.policedeptcount AS policedepartmentscount,
 ps.policeofficercount AS numpoliceofficerscount,
 hos.hospitalcount AS hospitalscount,
 fs.firestationcount AS firestationscount,
 fs.firefightercount AS firefighterscount,
 cs.childcarecount AS childcarecenterscount
FROM ZipCodesInCityOrState z
LEFT OUTER JOIN LatestMarket lm ON z.zipcode = lm.zipcode
LEFT OUTER JOIN IncomeStats ins ON z.zipcode = ins.zipcode
LEFT OUTER JOIN HealthStats hes ON z.zipcode = hes.zipcode
LEFT OUTER JOIN PoliceStats ps ON z.zipcode = ps.zipcode
LEFT OUTER JOIN HospitalStats hos ON z.zipcode = hos.zipcode
LEFT OUTER JOIN FireStats fs ON z.zipcode = fs.zipcode
LEFT OUTER JOIN ChildcareStats cs ON z.zipcode = cs.zipcode
WHERE COALESCE(lm.medianlistingprice, 0) >= $4
 AND COALESCE(lm.medianlistingprice, 0) <= $5
 AND COALESCE(ins.meanincome, 0) >= $6
 AND COALESCE(ins.meanincome, 0) <= $7
 AND COALESCE(hes.healthratio, 1) <= $8
 AND COALESCE(ps.policedeptcount, 0) >= $9
 AND COALESCE(ps.policeofficercount, 0) >= $10
 AND COALESCE(hos.hospitalcount, 0) >= $11
 AND COALESCE(fs.firestationcount, 0) >= $12
 AND COALESCE(fs.firefightercount, 0) >= $13
 AND COALESCE(cs.childcarecount, 0) >= $14
 AND COALESCE(z.population, 0) >= $15
 AND COALESCE(z.population, 0) <= $16
ORDER BY z.zipcode
LIMIT $17;
```

Materialized View Code for Optimization:

```
-- create materialized view for Zipcode Summary
CREATE MATERIALIZED VIEW market2022_view AS
SELECT lm.zipcode,
       AVG(lm.MedianListingPrice) as medianlistingprice
FROM localmarket lm
WHERE lm.monthdate like '2022%'
GROUP BY lm.zipcode ;

-- Create index on view to improve query performance
CREATE INDEX idx_2022market_view ON market2022_view(zipcode);
```

Query 2 (complex): /api/v1/analytics/city-aggregate-comparison

Description: Aggregate all housing, police, childcare, hospitals and other data for all ZIP codes within a city.
Pre-optimized: Running time 4 seconds
Post-optimized: Running time 400 ms
Optimized Query Code:

```
SELECT city,
      state,
      totalpopulation,
      meanincome,
      listingprice,
      firefighterdepartments AS numfiredept,
      firefighters AS numfirefighters,
      policestations AS numpolicedept,
      numchildcarecenters AS numchildcarecenter,
      numhospitals AS numhospitals,
      obesityrate AS obesityrate,
      asthmarate AS asthmarate,
      depressionrate AS depressionrate
FROM similar_city_view SC
WHERE LOWER(SC.city) = LOWER($1) AND LOWER(SC.state) = LOWER($2)
```

Query 3 (complex): /api/v1/analytics/similar-cities/:city/:state

Description: Find two similar cities from the input city that was used in the city aggregate comparison.
Pre-optimized: Running time 16 seconds
Post-optimized: Running time 500 ms
Optimized Query Code:

```
With current_city AS (
        SELECT *
        FROM similar_city_view SC
        WHERE LOWER(SC.city) = LOWER($1) AND LOWER(SC.state) = LOWER($2)
)


SELECT
   az.city,
   az.state,
   ROUND(
          ABS(COALESCE(az.TotalPopulation, 0) - COALESCE(cz.TotalPopulation, 0)) * 0.05 +
          ABS(COALESCE(az.MeanIncome, 0) - COALESCE(cz.MeanIncome, 0)) * 0.1 +
          ABS(COALESCE(az.listingprice, 0) - COALESCE(cz.listingprice, 0)) * 0.1 +
          ABS(COALESCE(az.PoliceStations, 0) - COALESCE(cz.PoliceStations, 0)) * 0.05 +
          ABS(COALESCE(az.PoliceOfficers, 0) - COALESCE(cz.PoliceOfficers, 0)) * 0.05 +
```

```
          ABS(COALESCE(az.NumHospitals, 0) - COALESCE(cz.NumHospitals, 0)) * 0.1 +
          ABS(COALESCE(az.NumChildcareCenters, 0) - COALESCE(cz.NumChildcareCenters, 0)) *
0.1 +
          ABS(COALESCE(az.FirefighterDepartments, 0) - COALESCE(cz.FirefighterDepartments,
0)) * 0.05 +
          ABS(COALESCE(az.Firefighters, 0) - COALESCE(cz.Firefighters, 0)) * 0.05 +
          ABS(COALESCE(az.AsthmaRate, 0) - COALESCE(cz.AsthmaRate, 0)) * 0.05 +
          ABS(COALESCE(az.ObesityRate, 0) - COALESCE(cz.ObesityRate, 0)) * 0.05 +
          ABS(COALESCE(az.DepressionRate, 0) - COALESCE(cz.DepressionRate, 0)) * 0.05
   ) AS similarity_score
FROM similar_city_view az
       CROSS JOIN current_city cz
ORDER BY similarity_score ASC
offset 1 rows
   fetch first 2 rows only;
```

Query 4 (complex): api/v1/analytics/similar-zipcodes/:zipcode

Description: Finds the 4 most similar zip codes based on a weighted calculation for the given zip code.

Pre-optimized: Very expensive query (8-10 s) since it contains multiple joins (including cartesian), aggregations, CTEs (where listingmarket has +1.2 million rows), and sorting. Memory usage was nearly 10 MB.

Explain Analyse (Raw) Summary:

Total cost: ~97 billion
Planning Time: 52.884 ms
Execution Time: 7292.046 ms

Post-optimized: After optimization the query ran very fast (100-600 ms), significant improvement from ~8+ seconds. The following changes helped achieve this:

- **Indexing:** Setting an index on zipcode lowered the query by 2-4 seconds by reducing the join and .
- **Materialized View:** Creating a materialized view on all zipcodes helped lower the query substantially to < 1 second since expensive joins, aggregations, and CTEs were saved on memory. The query could now reference the view and can perform comparisons against a specific zip code in a single pass. This helped lower memory to 25 kB, significantly lower than 10 MB of unoptimized query.
- **Possible drawbacks:** The materialized view needs to be refreshed if data is to be updated. However, for our project the data is static which makes views a good choice.

Explain Analyse (Raw) Summary:

Total cost: ~10k
Planning Time: 0.237 ms
Execution Time: 147.178 ms

Optimized Query Code:

```
-- Improved query (less than 1 second) --
WITH current_zipcode AS (
 SELECT *
 FROM all_zipcodes
 WHERE ZipCode = '19104'
)

SELECT
 az.ZipCode,
 ROUND(
   ABS(COALESCE(az.Population, 0) - COALESCE(cz.Population, 0)) * 0.05 +
   ABS(COALESCE(az.MeanIncome, 0) - COALESCE(cz.MeanIncome, 0)) * 0.1 +
   ABS(COALESCE(az.avgmedianlistingprice, 0) - COALESCE(cz.avgmedianlistingprice, 0)) * 0.1 +
   ABS(COALESCE(az.PoliceStations, 0) - COALESCE(cz.PoliceStations, 0)) * 0.05 +
   ABS(COALESCE(az.PoliceOfficers, 0) - COALESCE(cz.PoliceOfficers, 0)) * 0.05 +
   ABS(COALESCE(az.Hospitals, 0) - COALESCE(cz.Hospitals, 0)) * 0.1 +
   ABS(COALESCE(az.ChildcareCenters, 0) - COALESCE(cz.ChildcareCenters, 0)) * 0.1 +
   ABS(COALESCE(az.FirefighterDepartments, 0) - COALESCE(cz.FirefighterDepartments, 0)) * 0.05
+
   ABS(COALESCE(az.Firefighters, 0) - COALESCE(cz.Firefighters, 0)) * 0.05 +
   ABS(COALESCE(az.AsthmaRate, 0) - COALESCE(cz.AsthmaRate, 0)) * 0.05 +
   ABS(COALESCE(az.ObesityRate, 0) - COALESCE(cz.ObesityRate, 0)) * 0.05 +
   ABS(COALESCE(az.DoctorVisitsRate, 0) - COALESCE(cz.DoctorVisitsRate, 0)) * 0.05 +
   ABS(COALESCE(az.DepressionRate, 0) - COALESCE(cz.DepressionRate, 0)) * 0.05 +
   ABS(COALESCE(az.HousingInsecurityRate, 0) - COALESCE(cz.HousingInsecurityRate, 0)) * 0.05 +
   ABS(COALESCE(az.SocialIsolationRate, 0) - COALESCE(cz.SocialIsolationRate, 0)) * 0.05
 ) AS similarity_score
FROM all_zipcodes az
JOIN current_zipcode cz ON TRUE
WHERE az.ZipCode <> '19104'
ORDER BY similarity_score DESC
LIMIT 4;

---- MATERIALIZED VIEW ----
REFRESH MATERIALIZED VIEW all_zipcodes;


CREATE MATERIALIZED VIEW all_zipcodes AS (
   WITH firetable AS (
     SELECT
       ZipCode,
       SUM(ActivePersonnel) AS Firefighters,
       COUNT(DISTINCT DptId) AS FirefighterDepartments
     FROM FirefighterStations
     GROUP BY ZipCode
   ),

   policetable AS (
     SELECT
       ZipCode,
       SUM(TotalActiveOfficers) AS PoliceOfficers,
       COUNT(DISTINCT Id) AS PoliceStations
     FROM PoliceStations
     GROUP BY ZipCode
   ),

   listingtable AS (
       SELECT
           zipcode,
```

```
            AVG(medianlistingprice) as avgmedianlistingprice
        FROM localmarket
        GROUP BY zipcode
    )

    SELECT
        z.ZipCode,
        z.Population,
        hi.MeanIncome,
        lt.avgmedianlistingprice,
        pt.PoliceStations,
        pt.PoliceOfficers,
        COUNT(DISTINCT h.Id) AS Hospitals,
        COUNT(DISTINCT cc.Id) AS ChildcareCenters,
        ft.FirefighterDepartments,
        ft.Firefighters,
        MAX(CASE WHEN hm.Measure = 'Current asthma among adults'
            THEN hm.Ratio END) AS AsthmaRate,
        MAX(CASE WHEN hm.Measure = 'Obesity among adults'
            THEN hm.Ratio END) AS ObesityRate,
        MAX(CASE WHEN hm.Measure = 'Visits to doctor for routine checkup within the past year
among adults'
            THEN hm.Ratio END) AS DoctorVisitsRate,
        MAX(CASE WHEN hm.Measure = 'Depression among adults'
            THEN hm.Ratio END) AS DepressionRate,
        MAX(CASE WHEN hm.Measure = 'Housing insecurity in the past 12 months among adults'
            THEN hm.Ratio END) AS HousingInsecurityRate,
        MAX(CASE WHEN hm.Measure = 'Feeling socially isolated among adults'
            THEN hm.Ratio END) AS SocialIsolationRate
      FROM ZipCode z
      LEFT JOIN HouseholdIncome hi ON z.ZipCode = hi.ZipCode
      LEFT JOIN HealthMeasure hm ON z.ZipCode = hm.ZipCode
      LEFT JOIN policetable pt ON z.ZipCode = pt.ZipCode
      LEFT JOIN firetable ft ON z.ZipCode = ft.ZipCode
      LEFT JOIN Hospitals h ON z.ZipCode = h.ZipCode
      LEFT JOIN ChildcareCenters cc ON z.ZipCode = cc.ZipCode
      LEFT JOIN listingtable lt ON z.zipcode = lt.zipcode
      GROUP BY z.ZipCode, z.Population, hi.MeanIncome, pt.PoliceStations, pt.PoliceOfficers,
ft.FirefighterDepartments, ft.Firefighters, lt.avgmedianlistingprice
);
```

SIMPLE QUERIES

Query 5 (simple): /api/v1/analytics/city-growth-rate/:city/:state

Description: Population and economic growth trends

Query Code:

```
WITH CityGrowRateOld AS (
    SELECT
        Z.city,
        Z.state,
        Z.avglistingprice AS AvgListingPriceOld
    FROM housing2022_city_view Z
    WHERE Z.City ilike $1
      And Z.state ilike $2
```

```
),
    CityGrowRateNew AS (
        SELECT
            Z.city,
            Z.state,
            Z.avglistingprice AS AvgListingPriceNew
        FROM housing2025_city_view Z
        WHERE Z.City ilike $1
          And Z.state ilike $2
    )

SELECT
   CO.city,
   CO.state,
   ROUND(((AvgListingPriceNew - AvgListingPriceOld) / AvgListingPriceOld * 100), 2)::TEXT ||
'%' AS GrowthRate3Year
FROM CityGrowRateOld CO
        NATURAL JOIN CityGrowRateNew CN;
```

Query 6 (simple): /api/v1/search/featured-cities/location-based

Description: Provide search suggestions for cities, states, ZIP codes.

Query Code:

```
-- Step 1: Get cities with optimized filtering
WITH StateCities AS (
 SELECT DISTINCT
   z.city,
   z.state,
   z.population,
   z.zipcode
 FROM zipcode z
 WHERE LOWER(z.state) = LOWER($1::text)
   AND z.population >= 10000  -- Balanced threshold for good coverage
 ORDER BY z.population DESC
 LIMIT 50  -- Limit for geographic diversity
),
-- Step 2: Get all available data for these cities
CityData AS (
 SELECT
   sc.city,
   sc.state,
   sc.population,
   COALESCE(AVG(lm.medianlistingprice), 0) AS avg_listing_price,
   COALESCE(AVG(hm.ratio), 1) AS avg_health_measure
 FROM StateCities sc
 LEFT JOIN localmarket lm ON sc.zipcode = lm.zipcode
 LEFT JOIN healthmeasure hm ON sc.zipcode = hm.zipcode
```

```
 GROUP BY sc.city, sc.state, sc.population
),
-- Step 3: Calculate ranks and scores
RankedCities AS (
 SELECT
   city,
   state,
   population,
   avg_listing_price,
   avg_health_measure,
   ROW_NUMBER() OVER (ORDER BY avg_listing_price ASC) AS price_rank,
   ROW_NUMBER() OVER (ORDER BY avg_health_measure ASC) AS health_rank,
   (ROW_NUMBER() OVER (ORDER BY avg_listing_price ASC) +
    ROW_NUMBER() OVER (ORDER BY avg_health_measure ASC)) AS combined_rank
 FROM CityData
 WHERE avg_listing_price > 0 OR avg_health_measure < 1  -- Include cities with any meaningful
data
)
SELECT
 city,
 state,
 population,
 ROUND(avg_listing_price::numeric, 2) AS avg_listing_price,
 ROUND(avg_health_measure::numeric, 2) AS avg_health_measure,
 price_rank,
 health_rank,
 combined_rank
FROM RankedCities
ORDER BY combined_rank ASC
LIMIT $2::integer;
```

Query 7 (simple): /api/v1/analytics/location/:zipcode

Description: Get detailed information for a specific ZIP code.

Query Code:

```
WITH firetable AS (
 SELECT
   ZipCode,
   SUM(ActivePersonnel) AS Firefighters,
   COUNT(DISTINCT DptId) AS FirefighterDepartments
 FROM FirefighterStations
 GROUP BY ZipCode
),
policetable AS (
 SELECT
   ZipCode,
```

```
   SUM(TotalActiveOfficers) AS PoliceOfficers,
   COUNT(DISTINCT Id) AS PoliceStations
 FROM PoliceStations
 GROUP BY ZipCode
)
SELECT
 z.ZipCode,
 z.City,
 z.State,
 z.Population,
 hi.MeanIncome,
 lm.MedianListingPrice,
 lm.ActiveListingCount,
 pt.PoliceStations,
 pt.PoliceOfficers,
 COUNT(DISTINCT h.Id) AS Hospitals,
 COUNT(DISTINCT cc.Id) AS ChildcareCenters,
 ft.FirefighterDepartments,
 ft.Firefighters
FROM ZipCode z
LEFT JOIN HouseholdIncome hi ON z.ZipCode = hi.ZipCode
LEFT JOIN LocalMarket lm ON z.ZipCode = lm.ZipCode
LEFT JOIN policetable pt ON z.ZipCode = pt.ZipCode
LEFT JOIN firetable ft ON z.ZipCode = ft.ZipCode
LEFT JOIN Hospitals h ON z.ZipCode = h.ZipCode
LEFT JOIN ChildcareCenters cc ON z.ZipCode = cc.ZipCode
WHERE z.ZipCode = $1
GROUP BY z.ZipCode, z.City, z.State, z.Population,
        hi.MeanIncome, lm.MedianListingPrice, lm.ActiveListingCount,
        pt.PoliceStations, pt.PoliceOfficers,
        ft.FirefighterDepartments, ft.Firefighters,
        lm.monthdate
ORDER BY lm.monthdate DESC
LIMIT 1;
```

Query 8 (simple): /api/v1/real-estate/price-trends/:zipcode

Description: Historical price data for charts

Query Code:

```
SELECT
 monthdate,
 medianlistingprice
FROM localmarket
WHERE zipcode = $1
 AND medianlistingprice > 0
ORDER BY monthdate;
```

Query 9 (simple): /api/v1/health/community/:zipcode

Description: Health metrics and community wellness data

Query Code:

```
WITH hospitalTable AS (
 SELECT
   z.zipcode,
   COUNT(DISTINCT h.id) AS HospitalCount,
   CASE
     WHEN COUNT(DISTINCT h.id) > 0 AND z.population > 0 THEN ROUND(z.population /
COUNT(DISTINCT h.id), 2)
     ELSE 0
   END AS HospitalToPopulationRatio,
   CASE
     WHEN SUM(h.numofbeds) > 0 AND z.population > 0 THEN ROUND(z.population /
SUM(h.numofbeds), 2)
     ELSE 0
   END AS BedsToPopulationRatio
 FROM zipcode z
 LEFT JOIN hospitals h ON z.zipcode = h.zipcode
 GROUP BY z.zipcode, z.population
)
SELECT
 MAX(CASE WHEN hm.measure = 'Current asthma among adults' THEN hm.ratio END) AS AsthmaRate,
 MAX(CASE WHEN hm.measure = 'Obesity among adults' THEN hm.ratio END) AS ObesityRate,
 MAX(CASE WHEN hm.measure = 'Visits to doctor for routine checkup within the past year among
adults' THEN hm.ratio END) AS DoctorVisitsRate,
 MAX(CASE WHEN hm.measure = 'Depression among adults' THEN hm.ratio END) AS DepressionRate,
 MAX(CASE WHEN hm.measure = 'Housing insecurity in the past 12 months among adults' THEN
hm.ratio END) AS HousingInsecurityRate,
 MAX(CASE WHEN hm.measure = 'Feeling socially isolated among adults' THEN hm.ratio END) AS
SocialIsolationRate,
 ht.HospitalCount,
 ht.HospitalToPopulationRatio,
 ht.BedsToPopulationRatio
FROM zipcode z
LEFT JOIN healthmeasure hm ON z.zipcode = hm.zipcode
LEFT JOIN hospitalTable ht ON z.zipcode = ht.zipcode
WHERE z.zipcode = $1
GROUP BY z.zipcode, z.population, ht.HospitalCount, ht.HospitalToPopulationRatio,
ht.BedsToPopulationRatio;
```

Query 10 (simple): /api/v1/real-estate/affordable-zipcodes

Description: Finds affordable housing options
Query Code:

```
SELECT
 l.zipcode,
 l.medianlistingprice,
 l.monthdate,
 z.city,
 z.state
FROM localmarket l
JOIN zipcode z ON l.zipcode = z.zipcode
WHERE l.medianlistingprice <= $1::integer
 AND l.medianlistingprice > 0
 AND LOWER(z.city) = LOWER($2::varchar)
 AND LOWER(z.state) = LOWER($3::varchar)
ORDER BY l.medianlistingprice ASC
LIMIT $4::integer;
```

Query 11 (simple): /api/v1/search/autocomplete

Description: Taking a search term as input, this endpoint returns an array of location matches.
Query Code:

```
SELECT * FROM (
(SELECT z.zipcode AS value, 'ZipCode' AS matchType
FROM zipcode z
WHERE z.zipcode LIKE $1)
UNION ALL
(SELECT DISTINCT
  CONCAT(city, ', ', state) AS value,
  'City' AS matchType
FROM zipcode
WHERE CONCAT(city, ', ', state) ILIKE $2)
UNION ALL
(SELECT DISTINCT state AS value, 'State' AS matchType
FROM zipcode
WHERE state ILIKE $3)
)
ORDER BY
 CASE matchType WHEN 'State' THEN 1
   WHEN 'City' THEN 2
   WHEN 'ZipCode' THEN 3
   ELSE 4
 END,
 value
LIMIT $4
```

Appendix D: DDL for Table Creation

```
CREATE TABLE City (
        Name VARCHAR(255),  State VARCHAR(255),
        PRIMARY KEY (Name, State)
)

CREATE TABLE ZipCode (
        ZipCode VARCHAR(5) PRIMARY KEY CHECK (LENGTH(ZipCode) = 5),
        Latitude FLOAT NOT NULL, Longitude FLOAT NOT NULL,
        Population INT, City VARCHAR(255) NOT NULL,
        State VARCHAR(255) NOT NULL,
        FOREIGN KEY (City, State) REFERENCES City (Name, State)
)

CREATE TABLE HealthMeasure (
        Measure VARCHAR(255),          ZipCode VARCHAR(5),
        Ratio FLOAT NOT NULL CHECK (Ratio >= 0 AND Ratio <= 1),
        TotalPopulation INT NOT NULL,              PRIMARY KEY (Measure, ZipCode),
        FOREIGN KEY (ZipCode) REFERENCES ZipCode (ZipCode)
)

CREATE TABLE PoliceStations (
        Id INT PRIMARY KEY, ZipCode VARCHAR(5),
        Latitude FLOAT NOT NULL, Longitude FLOAT NOT NULL,
        PartTimeOfficers INT NOT NULL, FullTimeOfficers INT NOT NULL,
        TotalActiveOfficers INT NOT NULL,
        FOREIGN KEY (ZipCode) REFERENCES ZipCode (ZipCode)
)

CREATE TABLE Hospitals (
        Id INT PRIMARY KEY, ZipCode VARCHAR(5),
        Latitude FLOAT NOT NULL, Longitude FLOAT NOT NULL,
        NumOfBeds INT, TraumaLevel INT,
        HasHelipad BOOLEAN,
        FOREIGN KEY (ZipCode) REFERENCES ZipCode (ZipCode)
)

CREATE TABLE ChildcareCenters (
        Id INT PRIMARY KEY, ZipCode VARCHAR(5),
        Latitude FLOAT NOT NULL, Longitude FLOAT NOT NULL,
        FOREIGN KEY (ZipCode) REFERENCES ZipCode (ZipCode)
)

CREATE TABLE FirefighterStations (
        DptId INT PRIMARY KEY, ZipCode VARCHAR(5),
        NumberOfStations INT, ActivePersonnel INT,
        FOREIGN KEY (ZipCode) REFERENCES ZipCode (ZipCode)
)
```

```
CREATE TABLE LocalMarket (
        MonthDate VARCHAR(6),          ZipCode VARCHAR(5),
        ActiveListingCount INT,  MedianListingPrice INT,
        PRIMARY KEY (MonthDate, ZipCode),
        FOREIGN KEY (ZipCode) REFERENCES ZipCode (ZipCode)
)

CREATE TABLE HouseholdIncome (
        GeoID VARCHAR(255),            ZipCode VARCHAR(5),
        MeanIncome INT,          PRIMARY KEY (GeoID),
        FOREIGN KEY (ZipCode) REFERENCES ZipCode (ZipCode)
)
```

Appendix E: API Endpoints by Page

1. Homepage (/)

Featured Locations Section

Endpoint: GET /api/v1/search/featured-cities/location-based
Purpose: Get location-based featured cities based on user's IP
Response: Cities with population, prices, health metrics
Fallback: GET /api/v1/search/featured-cities (default cities)

Flow-1: Location-Based featured Cities (Primary Flow)
Sequence diagram (Link)

Flow-2: Default featured Cities (Alternative Flow)
Sequence diagram (Link)

Quick Search Categories

Endpoint: GET /api/v1/search/autocomplete
Purpose: Provide search suggestions for cities, states, ZIP codes
Parameters: term (search term), limit (result count)
Sequence Diagram (Link)

2. Search Results Page (/search-results)

ZIP Code Summaries

· Endpoint: GET /api/v1/search/zipcode-summaries
· Purpose: Comprehensive search with multiple filters
· Parameters:
  - state, city (location filters)
  - minPrice, maxPrice (price range)
  - healthMeasure, maxHealthRatio (health filters)

- minIncome, maxIncome (income filters)
- minPoliceDepts, minHospitals, minFireStations (facility filters)

Autocomplete for Search

· Endpoint: GET /api/v1/search/autocomplete

· Purpose: Real-time search suggestions

States and Cities

· Endpoint: GET /api/v1/search/states

· Purpose: Get all available states

· Endpoint: GET /api/v1/search/cities

· Purpose: Get cities by state

Sequence Diagrams

Autocomplete Search**:** Link

Page Initialization and data loading flow : Link

Filter updates and search: Link

View mode switch: Link

Error handling and loading states: Link

Zip code card interaction: Link

3. Location Details Page (/location-details/:zipcode)

Comprehensive Location Data

· Endpoint: GET /api/v1/analytics/location/:zipcode

· Purpose: Get detailed information for a specific ZIP code

· Data: Population, income, home prices, facilities, health metrics

Price Trends

· Endpoint: GET /api/v1/real-estate/price-trends/:zipcode

· Purpose: Historical price data for charts

· Response: Time series data with median listing prices

Similar ZIP Codes

· Endpoint: GET /api/v1/analytics/similar-zipcodes/:zipcode

· Purpose: Find similar locations based on characteristics

Community Health Data

· Endpoint: GET /api/v1/health/community/:zipcode

· Purpose: Health metrics and community wellness data

Sequence Diagrams

Page initialization and data loading: Link

Zip code data fetching: Link

City/State data fetching: Link

Price trends and similar zip codes: Link

City comparison features: Link

Error handling and loading states: Link

4. Comparison Page

City Comparison Data

· Endpoint: GET /api/v1/analytics/city-aggregate-comparison
· Purpose: Compare two cities across multiple metrics
· Parameters: city1, state1, city2, state2

Similar Cities

· Endpoint: GET /api/v1/analytics/similar-cities/:city/:state
· Purpose: Find cities with similar characteristics

Growth Rate Analysis

· Endpoint: GET /api/v1/analytics/city-growth-rate/:city/:state
· Purpose: Population and economic growth trends

Sequence Diagrams:

Page initialization and URL parameter parsing: Link
City/State comparison data fetching: Link
Growth rate data fetching: Link
Similar cities data fetching: Link
Zip code comparison data fetching: Link
Comparison analysis and rendering: Link
Error handling and loading states : Link

5. Real Estate Analysis

Affordable Housing

· Endpoint: GET /api/v1/real-estate/affordable-zipcodes
· Purpose: Find affordable housing options
· Parameters: city, state, maxPrice, limit

Market Trends

· Endpoint: GET /api/v1/real-estate/price-trends/:zipcode
· Purpose: Historical price analysis

6. Health Analysis

Health Measures

· Endpoint: GET /api/v1/health/measures
· Purpose: Available health metrics
· Endpoint: GET /api/v1/health/community/:zipcode
· Purpose: Community health data

7. Facilities Analysis

Facility Data

· Endpoint: GET /api/v1/facilities/search

- Purpose: Search for facilities by type and location
- Parameters: facilityType, city, state, zipcode